

Scheduling jobs

A new job can be scheduled by using the command `at <TIMESPEC>`. `at` will then read the commands to execute the `stdin`. For larger commands, and those that are case-sensitive, it is often easier to use input redirection from a script's file, e.g., `at now +5 min <myscript`, rather than typing all the commands manually in a terminal window. When entering commands manually, you can finish your input by pressing `CTRL+D`.

`<TIMESPEC>` allows for many powerful combinations, giving users an (almost) free-form way of describing exactly when a job should be run. Typically, they start with a time, e.g., 02:00pm, 15:59, or even teatime, followed by an optional date or the number of days in the future.

Some examples of combinations that can be used are listed in the following text. For a complete list, see the *timespec* definition in the *Reference* section (see note below).

 **Note:** Reference section for *crond* includes three man pages – *crond(8)*, *crontab(1)*, *crontab(5)*, which can be opened by the *man* command in the Red Hat Enterprise Linux 7 section.

```
now +5min
teatime tomorrow(teatime is 16:00)
noon +4 days
5pm august 3 2016
```

Inspecting and managing jobs

Inspecting jobs: Running the *atq* command will give a job scheduled to run in the future, a number, date and time schedule, as well as the queue for the job and its owner.

 **Note:** Normal, unprivileged users can only view and control their own jobs. The root user can view and manage all jobs.

To inspect the actual commands that will run when a job is executed, use the command `at-c <JOBNUMBER>`. The output will first show the environment for the job being set up to reflect the environment of the user who created the job (at the time this job was created), followed by the commands to be run.

Removing jobs: The command *atrm* `<JOBNUMBER>` will remove a scheduled job. This is useful when a job is no longer needed—for example, when a remote firewall configuration has succeeded and does not need to be reset.

Scheduling recurring jobs with Cron: Using *at*, one could, in theory, schedule a recurring job by having the job resubmit a new job at the end of its execution. In practice, this turns out to be a bad idea. RHEL systems ship with the *crond* daemon enabled and are started, by default, specifically for recurring jobs. *crond* is controlled by multiple configuration files, one per user (edited with the *crontab* (1) command),

and system wide files. These configuration files give users and administrators fine-grained control over exactly when their recurring jobs should be executed. The *crond* daemon is installed as part of the *crontab* package.

If the commands that run from a *crontab* job produce any output to either *stdout* or *stderr* that is not redirected, the *crond* daemon will attempt to email that output to the user owning that job (unless overridden) using the mail server configured on the system. Depending on the environment, this may need additional configuration.

Scheduling jobs: Normal users can use the *crontab* command to manage their jobs. This command can be called in four different ways:

```
crontab -l (List the jobs for the current user).
crontab -r (Remove all jobs for the current users).
crontab -e (Edit jobs for the current user).
crontab <filename> (Remove all jobs, and replace with the jobs read from <filename>. If no file is specified, stdin will be used.
```

 **Note:** The root can use the option `-u <username>` to manage the jobs for another user. It is not recommended to use the *crontab* command to manage system jobs; instead, the methods described in the next section should be used.

Job format

When editing jobs with *crontab-e*, an editor will be started (*vi* by default, unless the *EDITOR environment variable* has been set to something different). The file being edited will have one job per line. Empty lines are allowed, and comments start their line with a hash symbol (#). Environment variables can also be declared, using the format *NAME=value*, and will affect all lines below the line in which they are declared. Common environment variables in a *crontab* include *SHELL* and *MAILTO*. Setting the *SHELL* variable will change which shell is used to execute the commands on the lines below it, while setting the *MAILTO* variable will change which email address output will be mailed to.

 **Important:** Sending emails may require additional configuration of the local mail server or an SMTP relay on a system.

Individual jobs consist of six fields, which detail when and what should be executed. When all five of the first fields match the current date and time, the command in the last field will be executed. The fields are in the following order:

```
Minutes
Hours
Day-of-month
Month
Day-of-week
Command
```